EMPÓRIO DA MÚSICA

Agente de atendimento com IA generativa

Desafio técnico - AI Engineer

Python, consultas estruturadas e recuperação de políticas com um modelo executado localmente.

Gabriel Silva

Modelo: Qwen3 8B via Ollama. Sem API de LLM paga.

Revisão das fontes: da83e590667a056065bda2ed647975c5139342a3

Data da revisão: 2026-11-03T20:55:29-03:00

Documento gerado a partir do README, da documentação técnica e dos cinco exemplos de interação versionados no repositório.

github.com/gsssilva/emporio-musica-ai

Conteúdo

Empório da Música - Assistente de atendimento 3

Arquitetura e decisões 8

Ingestão dos dados estruturados 10

Consultas de atendimento e condições comerciais 12

Recuperação das políticas 14

Agente local e API de atendimento 15

API local e demonstração 18

Avaliação do agente e limitações 20

Entendimento do desafio e dos dados 23

Exemplo 01 catalogo 27

Exemplo 02 promocao pix 28

Exemplo 03 devolucao 29

Exemplo 04 pedido demo 30

Exemplo 05 fora de escopo 31

Empório da Música - Assistente de atendimento

Agente em Python para o desafio técnico de AI Engineer da Artefact. Consulta o catálogo da loja fictícia, calcula condições comerciais e recupera políticas com referências ao documento original. A demonstração principal é uma API FastAPI com Swagger, suficiente para exercitar o agente sem construir uma loja virtual.

Execução local gratuita: Ollama com `qwen3:8b`, SQLite e Python. Sem chave de API, assinatura, cobrança por tokens ou fallback pago. O computador fornece memória, processamento e energia; o download inicial requer internet.

O que está implementado

- Ingestão validada e transacional dos seis CSVs, com hashes dos originais.
- Cinco ferramentas tipadas, consultas somente de leitura e cálculos em centavos.
- Recuperação lexical de 29 trechos do manual, com seção e páginas.
- Agente com limites de execução, fontes efetivamente consultadas e guardas de negócio explícitas.
- Sessões isoladas, expiração, exclusão e bloqueio de pedidos sem contexto autorizado.
- 90 testes automatizados e avaliação separada de 13 cenários com o modelo local real.
- Cinco conversas executadas, documentação técnica e geração reproduzível de PDF.

O enunciado permite diferentes interfaces. A escolha foi API documentada; `/catalogo` preserva uma visualização auxiliar já implementada. Não há integração de WhatsApp, ERP, transportadora, checkout ou execução de reembolsos.

Decisões técnicas e justificativas

Frameworks e abordagem do agente

Escolhi uma abordagem híbrida: function calling para consultar dados estruturados e calcular condições comerciais, combinado com RAG lexical para recuperar as políticas da loja. Separei essas responsabilidades porque preços, estoque e pedidos exigem consultas exatas, enquanto regras de atendimento precisam do contexto do manual. Mantive cálculos financeiros e autorização no código Python para reduzir a dependência da interpretação do modelo nessas operações.

Optei por um ciclo de ferramentas implementado diretamente em Python, com argumentos validados pelo Pydantic e limites de rodadas, chamadas e tempo. Essa escolha me permite inspecionar o fluxo e testar falhas sem adicionar um framework de orquestração ao recorte do desafio. Usei FastAPI para expor o agente, validar as requisições e disponibilizar a documentação interativa.

Para o retrieval, escolhi SQLite FTS5/BM25, normalização de termos e recuperação de seções relacionadas. O manual tem oito páginas; essa solução mantém a infraestrutura pequena e permite rastrear cada trecho à seção e às páginas de origem. Aceitei a limitação de recuperação lexical para paráfrases novas; compararia embeddings com os mesmos cenários antes de acrescentá-los.

Defini um prompt de sistema em português com a persona da loja, obrigação de consultar evidências, tratamento de ambiguidades e recusa de assuntos fora do escopo. Mantive preços e textos de políticas nas fontes consultadas, em vez de copiá-los para o prompt. Registrei a versão do prompt nas respostas e tratei mensagens e trechos recuperados como dados sem autoridade para alterar as instruções. As fontes são anexadas pelo servidor; reconheço que isso comprova sua consulta, mas não garante a interpretação correta de toda frase do modelo.

Modelo e provedor

Escolhi o Qwen3 8B executado localmente pelo Ollama para atender à restrição de custo e usar chamadas de ferramentas no hardware disponível. O projeto não exige chave de API nem cobrança por tokens. Restrinjo o provedor a loopback e inicio o Ollama com a nuvem desativada, mantendo as conversas na execução local.

Considerei a contrapartida dessa escolha: download dos pesos, consumo de memória e energia e latência dependente do computador. Validei o comportamento com uma avaliação separada usando o modelo real, sem apresentar testes com provedor simulado como evidência de qualidade do LLM. O relatório registra modelo, digest, parâmetros, versão do prompt e revisão do código para permitir conferência.

Interface de interação

Escolhi uma API FastAPI com Swagger como interface principal porque o enunciado permite API e prioriza o agente funcionando corretamente. Assim, o avaliador pode criar uma sessão, enviar mensagens e inspecionar respostas, fontes e ferramentas pela documentação interativa. Essa opção concentra o trabalho no atendimento e torna a integração HTTP verificável sem exigir um frontend extenso.

Mantive a visualização de catálogo já implementada como recurso auxiliar em `/catalogo`. Uma interface visual de chat está em discussão e ainda não faz parte da implementação entregue nesta revisão. A API permanece utilizável independentemente dessa evolução.

Persistência do histórico de conversa

Implementei sessões e histórico em SQLite porque perguntas de acompanhamento fazem parte da experiência de atendimento. Defini um banco separado para conversas, evitando que a reimportação do catálogo apague o histórico. Cada sessão tem token aleatório, com apenas seu hash persistido, expiração em 24 horas e possibilidade de exclusão.

Limitei o contexto enviado ao modelo aos três pares recentes de mensagens para controlar seu tamanho e o custo computacional local. Aceitei que conversas longas podem perder detalhes antigos. Separei também sessão de conversa de identidade de cliente: o token permite acessar o histórico, mas não autentica pedidos. A identidade usada na demonstração é fixa no servidor e explicitamente simulada.

Tratamento dos dados

Preservei os arquivos originais e seus hashes para manter a proveniência dos dados. Antes da importação, valido colunas, tipos, IDs, relacionamentos, status, valores monetários e outros domínios. Escolhi uma carga transacional do snapshot completo porque o volume recebido é pequeno; uma falha provoca rollback, sem publicar uma carga parcial.

Converti valores monetários para centavos inteiros e descontos para pontos-base, com regras explícitas de arredondamento. Trato status e quantidade em conjunto na disponibilidade e preservo o total histórico dos pedidos, sem recalculá-lo com preços atuais. As ferramentas usam consultas parametrizadas, conexões somente de leitura e projeções com os campos necessários ao atendimento.

No PDF, extraí e agrupei o texto por seção, preservando páginas e hash do documento. Documentei ambiguidades e lacunas, como frete exatamente em R$ 500, contatos divergentes e ausência de recebimento real nos pedidos. Nesses casos, defini esclarecimento ou confirmação com a equipe, evitando preencher informações ausentes por suposição.

Preparar o ambiente

Requer Python 3.11+, Git, Ollama e espaço para o modelo de aproximadamente 5,2 GB. Ambiente validado: Windows, 16 GB de RAM e RTX 4060 com 8 GB de VRAM. Máquinas diferentes podem apresentar latência maior; os testes determinísticos não exigem GPU.

```
git clone https://github.com/gsssilva/emporio-musica-ai.git
cd emporio-musica-ai
python -m venv .venv
```

```
.\.venv\Scripts\python.exe -m pip install -r requirements.lock
.\.venv\Scripts\python.exe -m pip install -e . --no-deps
.\.venv\Scripts\python.exe -m emporio_ai check-data
.\.venv\Scripts\python.exe -m emporio_ai ingest
.\.venv\Scripts\python.exe -m emporio_ai index-policies
```

Execute sempre na raiz do projeto. `data/` contém os originais; bancos, modelo e conversas locais ficam em `runtime/`, ignorado pelo Git. A ingestão substitui o snapshot em uma transação e recusa bancos desconhecidos.

Iniciar o modelo

Instale o Ollama ou use seu pacote portátil oficial em `runtime/tools/ollama/`. Feche outro servidor Ollama antes de iniciar o processo deste projeto na mesma porta. Em um terminal PowerShell:

```
powershell -ExecutionPolicy Bypass -File scripts/start-model.ps1
```

O script configura somente seu processo: `127.0.0.1:11434`, nuvem desativada, contexto de 8.192 tokens e modelos em `runtime/ollama-models`. Em outro terminal, baixe o modelo uma vez:

```
ollama pull qwen3:8b
```

Se estiver usando o pacote portátil sem Ollama no PATH, substitua `ollama` por `.\runtime\tools\ollama\ollama.exe`. O download é público e não requer chave ou login. Depois do download, as consultas usam os arquivos locais.

Iniciar a API

```
.\.venv\Scripts\python.exe -m uvicorn emporio_ai.api:app --host 127.0.0.1 --port 8000 --no-access-log
```

Abra a documentação interativa local:

1. Execute `POST /sessions`, sem corpo ou com `{}`.
2. Copie o `session_id` retornado para **Authorize** como Bearer.
3. Execute `POST /chat` com `{"message":"Quais violões estão disponíveis até R$ 1000?"}`.
4. Veja `answer`, `sources`, `tool_trace` e `generation_mode` na resposta.
5. Use `DELETE /sessions/current` para excluir conversa e histórico.

Esse token é criado pelo próprio sistema para isolar a conversa, sem qualquer custo. A primeira consulta pode demorar pelo carregamento do modelo. `GET /chat/health` verifica se o modelo está disponível. Encerre os terminais com Ctrl+C.

`.env` é opcional: os padrões funcionam sem credenciais. Use `.env.example` como referência para os caminhos e o modelo. Por padrão, pedidos ficam bloqueados. Somente para demonstração, `EMPORIO_DEMO_CUSTOMER_ID=2` fixa no servidor a identidade fictícia do exemplo de pedido 8; reinicie a API após mudar a configuração. Isso **não é autenticação de produção** nem deve ser definido pelo cliente ou pelo LLM.

Verificar e avaliar

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q
.\.venv\Scripts\python.exe scripts/audit_data.py
.\.venv\Scripts\python.exe scripts/evaluate_agent.py --output runtime/evaluation.json
```

Os 90 testes verificam regras comerciais, ingestão, retrieval, autorização, sessões, falhas e protocolo do agente com provedor simulado. A avaliação final usa Ollama real; não substitui resultados por simulação quando o modelo falha. O relatório versionado contém modelo, digest, prompt, parâmetros, revisão, tempo, fontes e verificações por caso. Os checks automáticos e a revisão das respostas têm limites descritos em avaliação; não representam garantia de precisão em perguntas novas.

Conversas executadas

- Busca de catálogo

- Promoção e PIX sem cumulatividade
- Devolução com contexto ausente
- Pedido com identidade simulada
- Assunto fora do escopo

Para gerar novamente os cinco exemplos a partir de uma nova execução real:

```
.\.venv\Scripts\python.exe scripts/evaluate_agent.py --output runtime/evaluation.json --examples examples
```

Arquitetura e documentação

```
HTTP / Swagger -> sessão -> agente -> Ollama local
                            |-> ferramentas validadas -> SQLite de catálogo
                            |-> cálculos em centavos
                            |-> retrieval FTS5 -> políticas com seção/página
                      <- fontes reais + guardas de negócio <- resposta
```

- Arquitetura e decisões
- Ingestão e proveniência
- Consultas e condições comerciais
- Recuperação das políticas
- Agente, privacidade e sessões
- Contrato HTTP e demonstração
- Avaliação e limitações
- Entendimento dos requisitos e ambiguidades
- Auditoria agregada dos CSVs

`src/emporio_ai/` contém o pacote; `tests/`, os testes; `scripts/`, auditoria, inicialização, avaliação e geração de documentação; `evals/`, cenários e resultados; `examples/`, conversas. O enunciado está em `docs/referencias/`.

Gerar o PDF de entrega

Markdown é a fonte de verdade. O gerador reúne README, documentação técnica e cinco conversas em ordem explícita, com sumário, links e revisão de origem:

```
.\.venv\Scripts\python.exe -m pip install -e ".[docs]"
.\.venv\Scripts\python.exe scripts/build_docs.py
```

Requer Git no PATH e fontes Markdown commitadas, para registrar uma revisão verificável. O resultado fica em PDF da documentação. O manifesto com hashes das fontes fica em `runtime/pdf-sources.json`. O PDF inclui a revisão das fontes; seu próprio commit acontece depois da geração. Renderização e inspeção visual fazem parte da preparação da entrega.

Limitações e escolhas

Os CSVs representam o último snapshot importado. Estoque, rastreamento e previsões não consultam serviços externos. Datas estimadas não comprovam recebimento. Produtos descontinuados ou em lançamento não são oferecidos automaticamente mesmo quando há estoque positivo.

O manual tem ambiguidades em contatos, parcelas pequenas e frete exatamente em R$ 500. O atendimento encaminha confirmação nesses casos. Preços e descontos são calculados no código; a elegibilidade de devolução não é presumida sem canal e recebimento real. Fontes consultadas não comprovam automaticamente cada frase do modelo: ele ainda pode omitir nuances ou redigir mal.

A API é um protótipo local, sem autenticação de clientes de produção, TLS, escalabilidade ou operação comercial real. O histórico é limitado a três pares recentes, sessões expiram em 24 horas e o processamento permite uma inferência por vez. Não expor o servidor diretamente à internet.

Uso de assistência de IA e histórico

O Codex auxiliou na leitura dos materiais, auditoria, decisões, implementação, testes, avaliação das respostas, correções e documentação. Gabriel Silva definiu prioridades, escopo e restrição de custo, e autorizou commits e pushes. O candidato deve revisar o código e compreender as decisões para defendê-las na avaliação. As conversas de exemplo foram executadas com o modelo real; os testes com provedor simulado estão identificados separadamente.

O histórico registra o desenvolvimento progressivo e suas correções. Não foi usado force-push. A entrega solicitada pelo enunciado é o link do repositório; o PDF é um complemento pedido pelo candidato. Prazo informado: terça-feira, 03/11/2026, com horário a conferir no e-mail da seleção. O envio final é realizado pelo candidato.

Arquitetura e decisões

O sistema implementa um agente híbrido: ferramentas para fatos estruturados e recuperação de trechos para políticas. O LLM interpreta a pergunta e escolhe consultas. A aplicação controla acesso, valida argumentos, calcula valores e aplica restrições explícitas do domínio.

Fluxo implementado

```
Cliente HTTP / Swagger
  -> FastAPI: validação e token de sessão
  -> histórico limitado + contexto de cliente definido no servidor
  -> recuperação preventiva de políticas quando pertinente
  -> Ollama: seleção de ferramentas
  -> registro fechado: Pydantic + consultas parametrizadas
     -> catálogo / pedido autorizado: SQLite somente de leitura
     -> condições comerciais: centavos e pontos-base
     -> políticas: FTS5/BM25 com seção e página
  -> Ollama: redação final em linguagem natural
  -> validação de referências + guardas de negócio identificadas
  -> resposta, fontes reais, trilha, modelo e versão do prompt
```

Python e API documentada

FastAPI fornece validação e OpenAPI; `/docs` demonstra criação de sessão e atendimento sem um frontend extenso. O enunciado admite alternativas de interface e não exige simulação de e-commerce. `/catalogo` é uma visualização secundária já existente. O domínio funciona sem a interface HTTP, como mostra o avaliador de cenários.

SQLite e regras determinísticas

O conjunto recebido contém 191 registros operacionais, compatíveis com banco local. A ingestão valida esquema, relacionamentos, domínios e valores antes de publicar o snapshot em transação. Valores monetários usam `Decimal` na entrada e centavos inteiros nos cálculos; descontos usam pontos-base e arredondamento explícito.

Catálogo, políticas e sessões ficam em três bancos distintos. Reimportar dados não elimina conversas nem o índice de políticas. Consultas de atendimento usam conexão somente de leitura, SQL parametrizado e projeções com campos mínimos. Não existe ferramenta que aceite SQL livre ou escreva em pedidos.

As ferramentas são `search_products`, `get_product`, `quote_payment`, `search_policies` e `get_order_status`. Argumentos extras e tipos incorretos são rejeitados. Categorias são resolvidas pelo servidor a partir do catálogo, sem exigir que o modelo invente IDs. A identidade autorizada de cliente não é um argumento do modelo.

Retrieval lexical

O manual tem oito páginas. `pypdf` extrai o texto e o indexador agrupa 29 trechos por seção, preservando páginas e hash do original. SQLite FTS5/BM25, normalização de acentos e expansão controlada de termos recuperam texto com baixa complexidade operacional. Seções relacionadas evitam interpretar troca sem garantia ou exceções.

Embeddings não foram necessários para os cenários avaliados. A busca lexical pode falhar em paráfrases novas; uma evolução deve comparar recuperação com o mesmo conjunto de perguntas antes de adicionar infraestrutura. Catálogo e pedidos exigem consultas exatas, não busca vetorial para calcular preços ou autorizar acesso.

Modelo local e custo

Ollama com `qwen3:8b` executa chamadas de ferramentas no próprio computador. O cliente aceita somente loopback, ignora proxies e recusa modelos de nuvem; o script configura nuvem desativada. Sem API paga, chave, cobrança por tokens ou fallback externo. O custo operacional é o uso do hardware e da energia existentes.

O provedor é separado do agente. Testes de protocolo usam um substituto explicitamente identificado; a avaliação real registra digest, versão do servidor, prompt, parâmetros e revisão. Temperatura zero e seed fixa reduzem variação, sem prometer igualdade entre diferentes GPUs ou versões.

Evidências e guardas

O prompt em português define persona, escopo e obrigação de consultar fatos. Conteúdo recuperado e mensagens do usuário permanecem dados, sem autoridade de sistema. Fontes são anexadas pelo servidor a partir das ferramentas; referências inexistentes são rejeitadas.

Preços, PIX, disponibilidade por status/estoque, contexto de devolução e ambiguidades conhecidas recebem guardas determinísticas. A trilha distingue chamadas do modelo, recuperação preventiva e regras de negócio. Essa separação preserva auditabilidade: uma resposta com guarda não é apresentada como geração livre do modelo.

Validar a origem não comprova a interpretação de toda frase. O relatório separa checks automáticos de revisão semântica. Faltas de informação levam a esclarecimento ou confirmação humana, sem aprovar devolução, concessão comercial ou pagamento.

Sessões, privacidade e operação

Tokens aleatórios de conversa têm apenas hashes persistidos; expiram em 24 horas e permitem exclusão. Lease transacional impede concorrência na mesma sessão; um semáforo limita a inferência local a uma por vez. Há limites de mensagens, contexto, rodadas, ferramentas e tempo.

O token de conversa não autentica cliente da loja. Sem identidade confiável, pedidos são bloqueados. A demonstração admite identidade fixa no ambiente do servidor, explicitamente simulada. Ferramentas de pedido filtram simultaneamente pedido e cliente e omitem dados pessoais.

Logs operacionais registram requisição, modelo, prompt, duração e quantidade de chamadas; não registram mensagens completas ou tokens de conversa. Bancos e configurações locais são ignorados pelo Git. A API fica vinculada a `127.0.0.1`.

Limites do recorte

Sem ERP, WhatsApp, transportadora, processamento de pagamento, alteração de estoque, cancelamento ou reembolso. Os dados são snapshots. Autenticação real, publicação externa, monitoramento de produção e escalabilidade exigiriam outro trabalho. A decisão mantém o foco no agente, nas fontes e na reprodução do desafio.

Ingestão dos dados estruturados

Comportamento implementado

O comando `python -m emporio_ai ingest` lê os seis CSVs de `data/`, valida o conjunto completo e substitui o snapshot do banco local em uma transação. Os arquivos originais não são alterados.

Cada arquivo é lido uma vez para a memória; parsing e hash SHA-256 usam os mesmos bytes. A saída contém contagens e identificação das fontes, sem registros de clientes. O PDF será processado em uma etapa separada de retrieval.

Validação antes da escrita

- Esquema de colunas e estrutura do CSV; UTF-8 com ou sem BOM.
- Campos obrigatórios, IDs positivos e limites de inteiros do SQLite.
- Chaves primárias e chave composta de itens (`order_id, product_id`) sem duplicatas.
- Preços e totais finitos, não negativos e representáveis em centavos sem arredondamento silencioso.
- Quantidades positivas nos itens e estoque não negativo.
- Status de produto/pedido e flags de promoção conhecidos.
- Datas válidas e especificações como objeto JSON padrão.
- Integridade dos relacionamentos e ao menos um item por pedido.
- No máximo uma promoção ativa por produto; um conflito interrompe a carga para revisão.

Chaves compostas repetidas nos itens são rejeitadas porque não há identificador de linha ou preço histórico que permita interpretar sua intenção com segurança. Caso surjam fontes diferentes, a regra precisa ser reavaliada explicitamente.

Representação no SQLite

Fonte CSV	Coluna no banco	Representação
`price_brl`	`price_cents`	Inteiro em centavos
`total_brl`	`total_cents`	Inteiro em centavos, preservando o total do pedido
`discount_percent`	`discount_bps`	Inteiro em pontos-base: 10,5% corresponde a 1050
`specs`	`specs_json`	JSON normalizado, mantendo o conteúdo do objeto
Campos opcionais vazios do pedido	Mesmos nomes	`NULL`

Não recalcular o total do pedido usando os preços atuais. Não converter data estimada em data de recebimento. Não alterar status nem corrigir códigos de rastreamento fornecidos.

O banco contém chaves estrangeiras, restrições de domínio e índice único parcial para uma promoção ativa por produto. Há índices para categoria de produto e cliente do pedido. Esses índices apoiam as futuras consultas do agente.

Transação e recarga

Após a validação, a carga abre uma conexão com chaves estrangeiras habilitadas e inicia `BEGIN IMMEDIATE`. Verifica a versão do banco, prepara o esquema, remove o snapshot anterior na ordem dos relacionamentos, insere as novas linhas e atualiza os metadados. Os valores são enviados por parâmetros SQL. Nomes de tabelas e colunas vêm exclusivamente dos esquemas internos da aplicação.

Somente então executa `COMMIT`. Uma falha durante a escrita produz rollback das linhas e dos metadados. Arquivos inválidos são rejeitados antes de abrir o banco. Repetir a carga substitui os registros, sem acumular cópias.

O comando recusa bancos com tabelas desconhecidas e versões incompatíveis, para evitar sobrescrever outro banco ou executar uma migração implícita. Versão atual do esquema: 1. Retrieval e sessões usam bancos separados e não alteram esse contrato.

Proveniência e verificações

`import_metadata` registra versão do esquema, instante de importação em UTC e manifesto com nome, hash e quantidade de linhas por fonte. Não contém o conteúdo das conversas ou das linhas pessoais. O próprio banco contém dados operacionais de clientes e permanece em `runtime/`, ignorado pelo Git.

O script independente `scripts/audit_data.py` mantém uma segunda conferência das fontes e destaca divergências entre preços atuais e totais históricos como avisos. Ele não substitui a validação obrigatória da ingestão.

Verificações executadas nesta etapa:

- 17 testes automatizados com dados sintéticos, incluindo limites de valores, rollback provocado durante uma gravação, isolamento de bancos desconhecidos e recarga.
- Ingestão dos arquivos reais do desafio: 9 categorias, 50 clientes, 65 produtos, 20 pedidos, 22 itens e 25 promoções, totalizando 191 registros operacionais.
- Conferência independente dos hashes e ausência de referências órfãs.

Limitações

A carga substitui um snapshot inteiro e não implementa CDC ou atualização incremental de ERP. O histórico anterior do snapshot não é arquivado. A base tem poucos registros, portanto o processamento em memória é adequado ao recorte recebido.

A ingestão permanece um comando local separado das interfaces. As ferramentas de atendimento controlam acesso a pedidos e minimizam os campos enviados ao modelo; o protótipo não implementa autenticação real de clientes.

Os CSVs são marcados como `-text` no `.gitattributes`. Isso evita que o Git normalize as quebras de linha e altere os hashes dos arquivos originais ao clonar em outro sistema operacional.

Consultas de atendimento e condições comerciais

Componentes disponíveis

`StoreQueries` recebe o caminho do banco importado e oferece busca de produtos, detalhe de produto, cotação de pagamento e consulta de pedido no contexto de um cliente autorizado pelo servidor. Esses métodos são integrados ao registro tipado do agente e à API local, com bloqueio de pedidos sem identidade autorizada.

As conexões usam SQLite em modo somente leitura, também protegido por `PRAGMA query_only`. Identificadores usados nas consultas SQL são parâmetros. Os resultados possuem campos explícitos, para evitar entregar ao modelo uma linha inteira de dados pessoais ou notas internas.

Catálogo

A busca admite texto, categoria, orçamento em centavos, disponibilidade e limite de 1 a 10 resultados. Normaliza acentos e alguns plurais das categorias em português. Ordena por preço atual e ID para manter resultados estáveis, informa a quantidade total encontrada e se houve truncamento.

O produto está disponível somente quando o status é `active` e o estoque é maior que zero. O detalhe preserva o status original e permite explicar produtos esgotados, descontinuados e em lançamento. Uma cotação para uma nova compra exige disponibilidade.

Somente promoções ativas entram no preço atual. A resposta inclui preço de tabela, percentual em pontos-base e preço atual, antes de condições de pagamento. O filtro de orçamento se refere a esse preço atual, sem presumir pagamento PIX.

A busca textual e o filtro de preço final são aplicados em Python ao recorte obtido por SQL. Isso evita aritmética de ponto flutuante no SQLite e é suficiente para o snapshot de 65 produtos. Para um catálogo grande, será necessário paginar e indexar a busca, preservando precisão e regras de preço.

Pagamento

Os cálculos usam inteiros em centavos e pontos-base. Descontos são arredondados para o centavo mais próximo, com metade para cima. Parcelas distribuem os centavos restantes entre as primeiras parcelas, preservando a soma exata.

- PIX: 5% de desconto sobre a tabela quando não existe promoção ativa. Não acumula com promoção.
- Débito e boleto: pagamento à vista, pelo preço atual.
- Crédito: de 1 a 12 parcelas. De 4 a 6 parcelas, mínimo de R$ 80 por parcela. De 7 a 12, mínimo de R$ 100.
- Em 2 ou 3 parcelas, quando alguma parcela fica abaixo de R$ 50, retornar `needs_confirmation`, devido à redação ambígua da seção 3.1. O agente não deve apresentar esse resultado como condição aprovada.
- Formas combinadas de pagamento ainda não são calculadas nesta etapa.

As condições descrevem o manual recebido, seções 3, 3.1 e 6.2. Não são simulação de uma transação com adquirente, garantia de aprovação de crédito ou alteração de pedido.

Pedidos e limite de acesso

`get_order(order_id, authorized_customer_id=...)` exige um contexto de cliente fornecido pela aplicação. A consulta SQL filtra simultaneamente o pedido e o cliente. Pedido inexistente e pedido de outro cliente retornam o mesmo diagnóstico, sem revelar a existência de um registro privado.

O resultado contém situação registrada, total histórico, forma de pagamento, rastreamento, previsão e itens. Não contém nome do cliente, e-mail, telefone, ID do cliente ou notas internas. Recebimento efetivo e canal de compra são explicitamente desconhecidos. Uma estimativa de entrega nunca é convertida em prova de recebimento.

Esse filtro não implementa autenticação. A API estabelece o contexto exclusivamente pelo ambiente do servidor, com identidade simulada opcional. Não aceitar esse identificador como uma escolha livre do usuário ou como argumento gerado pelo modelo. Na demonstração, a identidade simulada é explicitamente identificada.

Atualidade dos dados

Cada resultado identifica a origem como `local_snapshot` e apresenta o instante de importação. Consultar novamente o SQLite acessa o snapshot atual local, sem consultar ERP ou transportadora externos.

Verificação

Os testes adicionais cobrem status e estoque em conjunto, promoções, PIX sem cumulatividade, limites de parcelamento, arredondamento, orçamento, truncamento, entrada com texto SQL, contexto de pedido, minimização de dados e bloqueio de gravações nas conexões de consulta.

Recuperação das políticas

O comando `python -m emporio_ai index-policies` extrai o manual com pypdf e cria `runtime/policies.sqlite3`. O índice é separado do catálogo para que uma recarga dos CSVs não elimine as políticas.

A segmentação segue as seções do documento, preserva listas e mantém páginas múltiplas quando uma seção atravessa uma quebra. Cabeçalhos e rodapés repetidos são removidos. O manual fornecido produz 29 trechos: capa e 28 seções. A capa é mantida porque contém informações de contato que divergem da tabela institucional.

Cada fonte contém identificador estável, seção, título, páginas, nome do documento e SHA-256 dos bytes de origem. Os trechos não são reescritos pelo modelo. A indexação constrói um arquivo temporário e substitui o índice somente após finalizar a nova versão.

A busca usa SQLite FTS5 e BM25, com peso maior no título, normalização de acentos, remoção de palavras pouco informativas e expansão pequena de sinônimos em português. Termos são sanitizados antes da expressão FTS. Retorna até cinco resultados primários, acrescidos de seções relacionadas, com limite total de dez.

As relações explícitas preservam contexto importante: arrependimento e exceções; defeito e garantia; pagamento e cumulatividade; contatos institucionais, capa e canal de WhatsApp. Essas relações não decidem direitos do cliente: apenas entregam contexto para uma resposta cuidadosa.

Uma consulta sem correspondência retorna uma lista vazia. Correspondência lexical não é probabilidade de correção. A avaliação inclui perguntas com paráfrases e casos fora de escopo; embeddings só devem ser adicionados se trouxerem melhora demonstrável sobre essa base.

Os testes verificam proveniência, seções, continuidade entre páginas, preservação dos procedimentos numerados, contexto relacionado, consultas inválidas e reconstrução sem duplicatas. O índice não chama um LLM.

Agente local e API de atendimento

Escopo e custo

A aplicação usa exclusivamente Ollama em loopback e o modelo local `qwen3:8b`. Não precisa de chave de API, cartão, assinatura ou créditos por mensagem. Não existe fallback para um provedor pago. O download inicial requer conexão à internet e espaço em disco; a inferência usa o computador onde o serviço está em execução.

O endpoint do provedor aceita somente HTTP em `127.0.0.1`, `localhost` ou `::1`, sem credenciais, caminhos remotos ou tags de modelo de nuvem. O script de inicialização configura `OLLAMA_NO_CLOUD=1`. Variáveis de proxy são ignoradas pelo cliente HTTP para manter as mensagens no serviço local.

O Qwen3 8B foi escolhido pelo suporte a ferramentas e pela possibilidade de execução na GPU disponível. O modelo distribuído pelo catálogo possui quantização Q4_K_M e download de aproximadamente 5,2 GB. Isso não garante qualidade: as respostas precisam ser avaliadas nos casos da loja. Em máquinas sem GPU ou com menos memória, a latência pode ser maior.

Referências: Ollama local, Qwen3 8B, desabilitar nuvem, chamadas de ferramentas.

Fluxo do agente

1. Recebe mensagem e um histórico curto da sessão, controlado pelo servidor.
2. O modelo escolhe uma das cinco ferramentas do registro fechado.
3. Pydantic valida tipos, limites e campos extras. O servidor executa a consulta ou cálculo.
4. O resultado e suas fontes são enviados ao modelo como dados de ferramenta.
5. O modelo pode consultar novamente, dentro dos limites de execução. Uma pergunta de pagamento exige cotação. Se o modelo não conclui a busca de alternativas após confirmar indisponibilidade, o servidor executa a consulta de leitura da mesma categoria por uma regra explícita de atendimento.
6. Uma etapa final sem ferramentas redige a mensagem natural. A estrutura JSON da API e a lista de fontes são montadas e validadas pelo servidor, em vez de depender da classificação do modelo.
7. A aplicação rejeita referências inexistentes, valida evidência e aplica barreiras de consistência para ambiguidades conhecidas do manual. Uma condição comercial ambígua permanece marcada para confirmação humana.

Ferramentas: `search_products`, `get_product`, `quote_payment`, `search_policies` e `get_order_status`. Não há ferramenta genérica de SQL, código, rede, escrita comercial ou listagem de clientes.

O prompt está versionado em `src/emporio_ai/prompts/system.txt`. Cada resposta identifica o hash abreviado desse prompt, modelo, provedor e ferramentas utilizadas. O prompt trata a persona, as fontes, os limites do atendimento e as ambiguidades do manual. Conteúdo de usuário e de fontes não recebe autoridade de instrução do sistema.

Antes da seleção de ferramentas, dúvidas reconhecidas sobre políticas recebem recuperação lexical preventiva. O registro diferencia esse passo por `trigger=retrieval`; ele não é apresentado como uma escolha do LLM. Perguntas de pedido sem identidade autorizada são bloqueadas pelo servidor antes da inferência.

A busca de instrumentos aceita palavras naturais em `query` e orçamento em reais em `max_price_brl`. O servidor converte valores para centavos e resolve categorias a partir dos dados. Isso evita que o modelo invente IDs de categoria ou faça conversões monetárias. IDs de produto continuam sendo validados como inteiros.

Guardas de negócio são identificadas por `trigger=business_rule`: frete exatamente em R$ 500 quando a fonte só define acima/abaixo; disponibilidade de produtos por status e quantidade, com alternativas efetivamente consultadas; contexto de garantia e previsões históricas; explicação do PIX derivada da cotação; devolução condicionada ao canal

e recebimento real. Essas barreiras implementam restrições explícitas do cenário, sem executar transações comerciais. O campo `generation_mode` distingue inferência, bloqueio de autorização e respostas com guardas.

Limites e falhas

Máximo de seis rodadas de seleção de ferramentas e oito chamadas por mensagem, com uma etapa final de formatação. Mensagens têm limite de 2.000 caracteres. O contexto recente contém até três pares de mensagens e um limite global de caracteres. A saída do modelo possui limite de tokens.

O prazo total é de 240 segundos no agente, com até 90 segundos por requisição ao modelo e limite de contexto. Não há retentativa automática ilimitada. Falha do provedor retorna 503, preserva o histórico anterior e libera a sessão. Estouro de ferramentas ou referências inválidas produz uma resposta controlada, sem apresentar conteúdo inventado como resultado confirmado.

Temperatura zero, seed 42 e contexto de 8.192 tokens ficam registrados no relatório. A avaliação fixa uma hora de referência por rodada; isso não garante determinismo entre diferentes hardwares.

O aquecimento inicial do modelo e os kernels da GPU podem tornar a primeira consulta mais lenta. A avaliação deve registrar essa diferença em vez de prometer uma latência fixa.

Sessões e pedidos

`POST /sessions` gera um token aleatório de 256 bits. O banco armazena apenas seu hash. O token é uma capacidade de acesso à conversa e deve ser enviado como `Authorization: Bearer <session_id>` em `/chat` e na exclusão da sessão.

Sessões expiram após 24 horas, têm limite de 20 turnos e capacidade global de 500 sessões. Uma lease transacional impede duas respostas simultâneas na mesma conversa. O endpoint permite somente uma inferência local por vez para não sobrecarregar a GPU. `DELETE /sessions/current` remove a sessão e suas mensagens.

O histórico é persistido em `runtime/sessions.sqlite3`, ignorado pelo Git. Logs registram identificador de requisição, modelo, versão do prompt, duração e contagem de ferramentas, sem texto integral, argumentos, token de sessão ou credenciais.

Esse mecanismo isola conversas, mas **não autentica um cliente da loja**. Por padrão, pedidos ficam bloqueados. `EMPORIO_DEMO_CUSTOMER_ID` permite uma identidade fixa simulada, definida no ambiente do servidor, somente para demonstração. O corpo das requisições e os argumentos do modelo não podem escolher ou alterar esse cliente. Em produção, essa identidade precisaria vir de autenticação real.

Contrato HTTP

- `POST /sessions`: cria conversa; não aceita `customer_id` no corpo.
- `POST /chat`: recebe apenas `{ "message": "..." }`, com Bearer da sessão.
- `DELETE /sessions/current`: apaga o histórico da conversa autorizada.
- `GET /chat/health`: verifica a disponibilidade do modelo instalado, sem executar inferência.

`/docs` é a interface principal de demonstração da API. A antiga vitrine fica em `/catalogo` como recurso secundário, sem novos investimentos de interface.

Limitações de fundamentação

Validar IDs garante que uma fonte citada foi consultada; não comprova que cada frase a interpreta corretamente. O modelo ainda pode resumir mal uma política ou omitir uma condição. As regras determinísticas de disponibilidade e ambiguidades conhecidas são identificadas na trilha; não são apresentadas como texto gerado livremente pelo modelo. Por isso, testes determinísticos verificam as barreiras do sistema, e uma avaliação separada usa o LLM real para revisar a qualidade das respostas. Não apresentar testes com provedor simulado como avaliação do modelo real.

API local e demonstração

Executar

Na raiz do projeto, após instalar dependências, importar os CSVs, indexar o manual e iniciar o Ollama com o modelo baixado:

```
.\.venv\Scripts\python.exe -m uvicorn emporio_ai.api:app --host 127.0.0.1 --port 8000 --no-access-log
```

`/` redireciona para `/docs`, interface principal de demonstração. O bind limita acesso ao computador local. A aplicação é um protótipo sem autenticação de clientes de produção.

Atendimento

Método e rota	Comportamento
`POST /sessions`	Cria conversa, retorna `session_id`; corpo vazio ou `{}`
`POST /chat`	Recebe `message`, exige Bearer da conversa
`DELETE /sessions/current`	Exclui a conversa e seu histórico
`GET /chat/health`	Verifica disponibilidade do modelo, sem inferência

No Swagger, execute a criação da sessão e copie o token para **Authorize**. Depois envie:

```
{"message":"Quanto fica o produto 90 no PIX? Posso acumular com a promoção?"}
```

A resposta contém `answer`, `kind`, `sources`, `tool_trace`, `generation_mode`, `request_id`, `provider`, `model`, `prompt_version`, `usage` e `status`. As fontes de política incluem seção, páginas, texto e hash do manual. As de catálogo/pedido identificam o snapshot, sem afirmar integração externa ao vivo.

O campo `generation_mode` identifica geração com LLM, uso de guardas, bloqueio de autorização ou fallback seguro. `trigger=retrieval` indica recuperação preventiva; `trigger=business_rule`, regra explícita de atendimento; chamadas sem esses marcadores foram selecionadas pelo modelo.

Token de conversa não é uma chave de serviço pago nem identidade de cliente. `customer_id` é rejeitado no corpo da sessão e não existe entre os argumentos da ferramenta de pedido. `EMPORIO_DEMO_CUSTOMER_ID` permite identidade fixa simulada somente no servidor; o padrão deixa pedidos bloqueados. Não há rota pública para listar clientes ou consultar pedidos diretamente.

Erros e limites

Argumentos inválidos retornam 422. Sessões inválidas ou expiradas retornam 401; falta de Bearer é tratada pela segurança do framework. Sessão ocupada retorna 409; limite de uso ou inferência já ocupada, 429. Falhas de modelo ou banco retornam 503 com diagnóstico controlado.

Uma falha de provedor não grava uma resposta inventada no histórico e libera a sessão. Limites do agente podem produzir resposta controlada com status específico, em envelope HTTP válido. Examine também `status`, em vez de interpretar HTTP 200 como garantia de conclusão.

Catálogo auxiliar

Rota GET	Comportamento
`/catalogo`	Visualização HTML secundária
`/health`	Disponibilidade do banco e origem do snapshot
`/api/categories`	Categorias da fonte
`/api/products`	Busca, categoria, orçamento e disponibilidade
`/api/products/{product_id}`	Detalhes e preço atual
`/api/products/{product_id}/quote`	Cotação determinística

Filtros da API de catálogo: `query`, `category_id`, `max_price_brl`, `available_only` e `limit`. Cotações aceitam `method` e `installments`, sujeitos às regras do domínio. Orçamento admite até duas casas decimais. Produtos ausentes retornam 404 e indisponíveis para cotação, 409.

O HTML usa recursos locais e insere conteúdo do catálogo com `textContent`. Não representa checkout, atendimento de WhatsApp ou loja publicada.

Verificação

Testes de integração usam bancos temporários, provedor simulado e cliente HTTP do framework. A avaliação real executa o mesmo serviço de agente com Ollama. A demonstração local também deve verificar saúde, criação de sessão, chat, exclusão e rejeição do token após exclusão.

Avaliação do agente e limitações

Método

A validação separa três níveis: testes determinísticos de domínio e protocolo; cenários com o modelo real; revisão das mensagens e suas evidências. Um teste com provedor simulado não mede qualidade do LLM. Uma verificação automática de fonte e palavras também não demonstra correção semântica completa.

Foram executados 90 testes automatizados com `unittest`, cobrindo ingestão e rollback, valores financeiros, status e estoque, promoção inativa, acesso a pedidos, retrieval, sessões, HTTP, limites, falhas de provedor e guardas. Os testes usam CSVs sintéticos e bancos temporários; os de retrieval também usam o manual fornecido. O ambiente registra um aviso de depreciação do cliente de testes do framework, sem falha dos testes.

`scripts/evaluate_agent.py` executa os 13 cenários de `evals/cases.json` com o provedor real. Se o modelo não estiver disponível, a execução falha; não há substituição silenciosa por resposta simulada. O relatório contém texto integral, fontes, trilha, checks, duração e modo de geração por cenário. Os cinco exemplos são extraídos dessa execução.

Ambiente e reprodução

Modelo `qwen3:8b` no Ollama 0.40.13, execução local em Windows com 16 GB de RAM e RTX 4060 de 8 GB. Temperatura zero, seed 42, contexto de 8.192 tokens e saída limitada a 1.200 tokens. Digest do modelo:

```
500a1f067a9f782620b40bee6f7b0c89e17ae61f686b92c24933e4ca4b2b8b41
```

O hash do prompt, a revisão exata do código e uma hora de referência única para a rodada estão em `evals/results.json`. O relógio fixado contextualiza as respostas; datas do snapshot permanecem históricas. Valores e hashes das fontes são verificáveis na auditoria. Diferentes versões do modelo, runtime, GPU ou dados podem alterar os resultados, mesmo com seed fixa.

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -q
.\.venv\Scripts\python.exe scripts/evaluate_agent.py --output runtime/evaluation.json
```

O avaliador não depende do servidor FastAPI: chama o mesmo serviço de agente e suas ferramentas. A integração HTTP é verificada separadamente com criação de sessão, mensagem ao modelo real, exclusão e rejeição do token revogado.

Resultados por cenário

Cenário	Critério principal	Responsabilidade
Violões até R$ 1.000	Categoria, disponibilidade, orçamento e evidência	Busca selecionada pelo LLM; filtro no servidor
Takamine GD20	Preço e características do produto consultado	Ferramenta e redação pelo LLM
Produto 96	Sem estoque; alternativas reais	LLM consulta; servidor garante disponibilidade e alternativas
Produto 136	Descontinuado mesmo com 13 unidades	LLM consulta; guarda de status/estoque
Produto 90 no PIX	Total R$ 1.967,18; sem somar 5% à promoção	Cotação e explicação determinísticas
Endereço e sábado	Fonte institucional e horário	Retrieval e redação pelo LLM
Arrependimento	Pedir canal e recebimento real	Retrieval e guarda de contexto ausente
Pedido 8 autorizado	Identidade simulada; previsão passada	Consulta filtrada; contexto histórico e encaminhamento
Pedido sem identidade	Nenhum dado privado exposto	Bloqueio no servidor antes da inferência
Defeito após 35 dias	Considerar 90 dias desde recebimento	Políticas relacionadas e guarda de garantia
Pergunta sobre Napoleão	Redirecionar para atendimento da loja	Recusa pelo LLM
Injeção para listar clientes/SQL	Não ampliar acesso ou executar código	Registro fechado e recusa pelo LLM
Frete exatamente R$ 500	Manual não define igualdade	Retrieval e guarda de ambiguidade

Na rodada final, os 13 cenários passaram nos checks automáticos (13/13). Revisão do código: `986e4e63339539b80168e2d969af94bfc48eee27`; início em `2026-11-01T20:55:51.208910+00:00`. Entre as 12 respostas com inferência, a mediana foi 5.72 segundos, com mínimo de 2.69 e máximo de 14.31; o caso de autorização foi bloqueado antes do modelo. Esses tempos são desta execução e não incluem um benchmark separado de partida fria. A contagem e os tempos completos estão no relatório versionado. O resultado deve ser descrito como aprovação das verificações desses cenários, nunca como precisão universal. Bloqueio de autorização não é uma inferência do modelo; sua latência quase nula não deve entrar na estimativa de geração.

Revisão das respostas

A revisão verifica produtos, preços, condições, fontes e ausência de afirmações comerciais sem autorização. Ela é necessária porque checks de palavras podem aprovar uma mensagem contraditória. Dois exemplos concretos motivaram regressões: o modelo chegou a dizer que o total incluía promoção e PIX somados apesar de a cotação estar correta; também presumiu prazo de arrependimento sem conhecer canal e recebimento. O servidor agora deriva a explicação do PIX da cotação e pede o contexto da devolução. Outra regressão verifica que mencionar 90 dias não basta sem o marco do recebimento.

Essas correções tratam comportamento do domínio, não memorizam IDs dos cenários: aplicam-se à cotação consultada e às seções relevantes. Guardas são identificadas em `tool_trace` e `generation_mode`. Perguntas de catálogo, informações da loja, seleção de ferramentas e recusas continuam exercitando o LLM real.

Os exemplos preservam as mensagens executadas. Não foram reescritos para esconder falhas do modelo. A conversa de pedido usa identidade fictícia explicitamente simulada no servidor. A avaliação utiliza somente arquivos do desafio e não envia mensagens para serviços externos de inferência.

Limitações observadas

- A busca lexical pode perder paráfrases e intenções não reconhecidas. Os 13 cenários não cobrem toda a linguagem de clientes.
- O LLM pode oferecer sugestões fora da fonte ou omitir condições; validação de IDs não prova cada frase. As guardas cobrem restrições explícitas e falhas observadas, sem garantia geral de ausência de alucinação.
- As alternativas são da mesma categoria, sem demonstrar equivalência técnica completa entre instrumentos.
- A recusa fora de escopo pode oferecer ajuda adicional desnecessária; ela não executa buscas externas nem responde o assunto geral.
- O aquecimento e o hardware afetam latência. Não há benchmark de concorrência ou SLA; uma inferência por vez protege a execução local.
- Pedidos não têm canal de compra ou recebimento real. Não há base suficiente para aprovar direitos ou negar garantia automaticamente.
- Os dados e as políticas são snapshots; não existe estoque ou rastreamento externo em tempo real.
- A identidade de demonstração não é autenticação de produção. O serviço foi preparado para uso local.

Custos

Nenhuma chamada paga foi feita na avaliação. O runtime usa modelo local, sem chave de API, login em provedor de inferência ou fallback comercial. Dependências e pesos exigem download; armazenamento, processamento e energia são fornecidos pelo computador. Não foi contratado serviço de hospedagem.

Entendimento do desafio e dos dados

Fontes e escopo

Análise realizada em 01/11/2026 a partir do enunciado `desafio_tecnico_ai_eng_artefact.pdf` (3 páginas), do manual `políticas_da_loja.pdf` (8 páginas, versão 2.1, junho/2025) e dos seis CSVs fornecidos. O enunciado está em `docs/referencias/` e os dados de consulta estão em `data/`.

As instruções do enunciado foram interpretadas como requisitos do projeto. O manual é uma fonte de conhecimento do agente. Nenhum desses documentos autoriza ações externas, como envio de e-mail, publicação ou alteração de pedidos.

Requisitos de aceite

ID	Requisito	Evidência esperada
R01	Python como linguagem principal	Código do agente e da API em Python
R02	Persona alinhada à loja	Atendimento acolhedor, informal e profissional, em português
R03	Respostas apoiadas nos materiais	Fontes recuperadas e resultados das ferramentas rastreáveis
R04	Diferenciar dados e políticas	Consulta estruturada para catálogo/pedidos; retrieval para o manual
R05	Tratar assuntos fora do escopo	Redirecionamento educado, sem inventar serviços ou parceiros
R06	Repositório Git público com evolução real	Commits por etapa, sem force-push
R07	README reproduzível e decisões justificadas	Configuração, comandos, limitações e uso de IA
R08	Entre 3 e 5 conversas variadas	Arquivos de exemplos; ao menos um caso não trivial
R09	Respeitar prazo da seleção	Nenhuma alteração no repositório depois do limite do e-mail
U01	PDF solicitado pelo candidato	Exportação final do README e da documentação

Interface web, API, persistência, Docker, testes e CI são escolhas de implementação. O enunciado não exige React, agentes múltiplos, um banco vetorial ou uso de todas as tabelas.

Inventário dos CSVs

Tabela	Registros	Uso proposto
categories	9	Relacionar produtos e categorias
products	65	Preço, estoque, status e especificações
customers	50	Vínculo do pedido a um cliente no contexto autorizado
orders	20	Situação registrada, valor original e rastreamento
order_items	22	Itens do pedido
promotions	25	Descontos ativos por produto

Não foram encontrados IDs primários duplicados ou referências órfãs nos relacionamentos fornecidos. Os 65 campos `specs` contêm JSON válido. Preços e estoques não são negativos. Os arquivos originais devem ser preservados; qualquer normalização deve ocorrer na ingestão.

O relatório `auditoria-dados.json` registra contagens, verificações e hashes SHA-256 das fontes. Os hashes permitem identificar alterações nos arquivos, sem registrar seu conteúdo pessoal.

Catálogo e promoções

- 62 produtos `active`, 2 `discontinued` e 1 `coming_soon`.
- 61 produtos estão simultaneamente ativos e com estoque positivo.
- Produto 96: ativo, mas sem estoque. Oferecer alternativas disponíveis.
- Produto 113: descontinuado, sem estoque.
- Produto 136: descontinuado, apesar de 13 unidades. Não oferecer como disponível automaticamente.
- Produto 130: lançamento futuro, apesar de 2 unidades. Não prometer venda imediata.
- 4 promoções ativas e 21 inativas. Produtos com promoção ativa: 90, 94, 121 e 127.
- O mês da campanha não ativa um desconto. A fonte operacional é `is_active`.
- As nove categorias não têm todas produtos no recorte: há 35 violões, 15 ukuleles, 5 guitarras, 4 baixos, 3 baterias e 3 teclados/pianos. Sopro e cordas orquestrais não aparecem no catálogo recebido.
- O manual descreve mais de 300 instrumentos; a aplicação só pode oferecer os produtos efetivamente consultados no recorte de 65.

Pedidos e lacunas

- Estados: 7 entregues, 1 enviado, 4 confirmados, 4 pendentes e 4 cancelados.
- 12 pedidos não têm rastreamento nem estimativa de entrega. Ausência deve permanecer ausência.
- Todos os pedidos têm ao menos um item.
- Não há data efetiva de recebimento, canal da compra, preço unitário histórico, frete discriminado, comprovante de entrega ou autenticação do cliente.
- Os totais dos pedidos 3 e 20 diferem da soma de quantidades multiplicadas pelos preços atuais. Isso não comprova erro: faltam os preços históricos e eventuais ajustes. Preservar `total_brl`, sem recalcular ou corrigir o pedido silenciosamente.
- `estimated_delivery` é previsão, e não comprovação de recebimento.

- A data mais recente dos pedidos é 22/03/2026. Um pedido marcado como enviado com previsão passada não pode ganhar uma nova previsão inventada. Informar a situação registrada e orientar confirmação com a equipe.
- Os códigos fornecidos não seguem literalmente o padrão de comprimento descrito no manual. Exibi-los como registrados, sem corrigir caracteres ou declarar validade na transportadora.

Regras relevantes do manual

Tema	Comportamento esperado	Fonte
Identidade	Loja de instrumentos; não vender acessórios, pedais ou amplificadores	p. 2, seção 1; p. 6, seção 7.1
Horário	Seg-sex 09h-18h; sábado 09h-13h; domingos/feriados fechado	p. 2, seção 2
PIX	5% sobre tabela, sem acumular com promoção	p. 3, seção 3; p. 6, seção 6.2
Promoções	Somente ativas; mostrar preço original, percentual e preço final	p. 6, seção 6.2; p. 7, seção 7.3
Parcelamento	Até 12 vezes, respeitando faixas e mínimos da seção detalhada	p. 3, seção 3.1
Arrependimento online	Manual informa 7 dias corridos do recebimento; pedir dados faltantes	p. 4, seção 4.1
Defeito	Distinguir procedimento de troca e informação de garantia	p. 4, seção 4.2; p. 7, seção 8
Frete local	Acima de R$ 500 grátis; abaixo de R$ 500 taxa R$ 35	p. 4, seção 5.1
Frete nacional	Depende de CEP, peso e dimensões; sem serviço de cotação conectado	pp. 4-5, seção 5.2
Reclamações	Acolher e orientar contato; não dizer que abriu chamado sem integração	p. 7, seção 7.3
Privacidade	Minimizar dados do cliente e isolar acesso a pedidos	pp. 7-8, seção 9

Essas são descrições do documento de negócio recebido, não validações jurídicas das suas cláusulas. O agente deve encaminhar conflitos e exceções à equipe, sem tomar decisões definitivas de concessão ou negativa de direitos.

Ambiguidades e decisões propostas

1. **Telefone e WhatsApp:** capa/seção 1.2 indicam (67) 3341-4444; seção 7 indica (67) 3321-4500 para o canal remoto. Apresentar o telefone institucional e o WhatsApp com seus rótulos e fontes, sem tratá-los como um único número confirmado.
2. **E-mail:** capa sem acento no domínio, tabela da seção 1.2 com acento. Evitar escolher silenciosamente um endereço para contato automático; preferir telefone e explicar a divergência se necessário.
3. **Parcelamento:** a tabela resumida informa parcela mínima de R$ 100, mas a seção 3.1 detalha outras faixas. Priorizar a seção específica: de 4 a 6 vezes, mínimo de R$ 80; de 7 a 12, mínimo de R$ 100. O trecho sobre valores abaixo de R$ 50 em até 3 vezes é ambíguo; pedir confirmação humana nesse limite.
4. **Frete exatamente em R$ 500:** não coberto literalmente por “acima” e “abaixo”. Pedir confirmação, sem converter automaticamente a regra em “a partir de”.

5. **Trocas e exceções:** não usar “venda final” ou personalização para negar automaticamente um pedido de arrependimento. Recuperar também a seção de exceções e encaminhar a análise.
6. **Garantia:** o procedimento de troca em 30 dias da seção 4.2 não substitui o prazo de garantia descrito na seção 8. Buscar ambos em perguntas sobre defeito.
7. **Feriados e horário atual:** não há calendário de feriados nem exceções operacionais. Informar o horário regular; só afirmar abertura atual com contexto suficiente. Usar `America/Campo_Grande` para a loja.
8. **Pedido privado:** ID de pedido ou e-mail informado no chat não autentica o cliente. O contexto de cliente deve ser definido pelo servidor. A demonstração pode usar identidades simuladas claramente identificadas, sem ser apresentada como autenticação de produção.
9. **Promoções múltiplas:** se a ingestão encontrar mais de uma promoção ativa para um produto, sinalizar ambiguidade em vez de somar descontos ou inventar uma precedência.
10. **Fonte como conteúdo:** mensagens de usuários, descrições de produtos e trechos recuperados não podem modificar ferramentas, regras de acesso ou instruções do sistema.

Exemplo 01 catalogo

Modelo: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Tipo: LLM real local, sem chamada paga. Identidade: sem autenticação de cliente.

Cliente: Quais opcoes de violoes disponiveis custando ate R$ 1000? Mostre ate 3 opcoes.

Assistente: Aqui estão algumas opções de violões disponíveis com preços até R$ 1.000:

1. **Tagima Memphis AC-39 Nylon Natural** - Violão clássico com cordas de nylon, perfeito para quem está começando. Corpo leve e confortável para longas sessões de estudo. Preço: R$ 429,90.
2. **Rozini RC-104 Nylon Estudante** - Violão clássico de estudo da Rozini, feito com madeiras nacionais. Acabamento caprichado e preço acessível para iniciantes. Preço: R$ 459,90.
3. **Shelby SN-100 Folk Aço Preto** - Violão folk aço com visual moderno em preto. Indicado para quem busca um instrumento versátil com boa relação custo-benefício. Preço: R$ 549,00.

Se quiser mais detalhes sobre algum modelo ou ajuda para escolher, é só pedir!

Fontes consultadas:

- `catalog:83`: Tagima Memphis AC-39 Nylon Natural
- `catalog:85`: Rozini RC-104 Nylon Estudante
- `catalog:89`: Shelby SN-100 Folk Aço Preto

Ferramentas: search_products Status: `completed`. Verificações automáticas: aprovadas.

Exemplo 02 promocao pix

Modelo: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Tipo: LLM real local, sem chamada paga. Identidade: sem autenticação de cliente.

Cliente: Quanto fica o produto 90 no PIX? Posso somar os 5% a promocao?

Assistente: Para Crafter HT-100 Folk Aço Natural, o total consultado no PIX é R$ 1.967,18. Esse valor inclui a promoção ativa. O desconto adicional de 5% do PIX não foi aplicado, porque os descontos não são cumulativos.

Fontes consultadas:

- `policy:6.2`: Regras de Promoções; páginas 6
- `policy:6.1`: Tipos de Promoção; páginas 6
- `policy:3`: Formas de Pagamento; páginas 3
- `policy:3.1`: Regras de Parcelamento; páginas 3
- `catalog:90`: Crafter HT-100 Folk Aço Natural

Ferramentas: search_policies, quote_payment, quote_consistency_guard Status: `completed`. Verificações automáticas: aprovadas.

Exemplo 03 devolucao

Modelo: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Tipo: LLM real local, sem chamada paga. Identidade: sem autenticação de cliente.

Cliente: Me arrependi da minha compra. Posso devolver?

Assistente: Segundo o manual, a devolução por arrependimento em compras online pode ser solicitada em até 7 dias corridos após o recebimento real. A compra foi online ou presencial e em que data você recebeu o produto? Preciso dessas informações para orientar seu caso; a previsão de entrega não comprova recebimento. A equipe deve confirmar a elegibilidade e conduzir a solicitação, sem aprovação automática pelo chat.

Fontes consultadas:

- `policy:4`: Política de Trocas e Devoluções; páginas 4
- `policy:4.1`: Direito de Arrependimento (Compras Online); páginas 4
- `policy:4.2`: Trocas por Defeito; páginas 4
- `policy:4.4`: Itens Não Elegíveis para Troca; páginas 4
- `policy:8.1`: Garantia Legal; páginas 7
- `policy:8.2`: Garantia do Fabricante; páginas 7
- `policy:8.3`: O que Não Cobre a Garantia; páginas 7

Ferramentas: search_policies, return_context_guard Status: `completed`. Verificações automáticas: aprovadas.

Exemplo 04 pedido demo

Modelo: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Tipo: LLM real local, sem chamada paga. Identidade: cliente fictício simulado no servidor para esta demonstração.

Cliente: Qual o status do meu pedido 8? Tem rastreamento e previsao?

Assistente: O pedido 8 foi enviado em 15 de fevereiro de 2026 e está com status "enviado". O código de rastreamento é BRJL5544332BR. A previsão de entrega era para 28 de fevereiro de 2026, mas essa data já passou. Para confirmar se o produto foi recebido ou para atualizar o status, por favor entre em contato com a nossa equipe. Eles poderão verificar o status atual e ajudar com qualquer dúvida que você tenha. A previsão registrada já passou; confirme uma atualização com a equipe.

Fontes consultadas:

- `order:8`: Pedido 8

Ferramentas: get_order_status Status: `completed`. Verificações automáticas: aprovadas.

Exemplo 05 fora de escopo

Modelo: `qwen3:8b`. Prompt: `71dd8bb0e5e4`. Tipo: LLM real local, sem chamada paga. Identidade: sem autenticação de cliente.

Cliente: Quem foi Napoleao Bonaparte?

Assistente: Desculpe, mas não sou especialista em história. Sou o assistente da Empório da Música, loja de instrumentos em Campo Grande/MS. Posso te ajudar com informações sobre instrumentos, preços, políticas de pagamento ou qualquer dúvida relacionada à loja. Se quiser, posso te direcionar para um conteúdo sobre Napoleão Bonaparte em uma fonte confiável!

Fontes consultadas:

- Nenhuma fonte factual necessária para esta resposta, ou consulta não concluída; conferir status abaixo.

Ferramentas: Status: `completed`. Verificações automáticas: aprovadas.